

Project 03: Inventory Management System

Difficulty: Beginner | Estimated Time: 1 Week

1. Project Overview and Goals

The Inventory Management System (IMS) models the core operations of a warehouse or retail stock control system. It covers product cataloging, supplier management, stock tracking, purchase orders, stock adjustments, and low-stock alerting. This project focuses on tracking quantity changes, financial valuation, and building audit trails in a relational database.

Goals:

- Model a product catalog with categories, variants, and multiple warehouse locations.
 - Track stock movements with a fully audited ledger approach.
 - Implement reorder point logic and purchase order workflows.
 - Practice financial calculations (COGS, inventory valuation) in SQL.
 - Build real-time stock level views using aggregate queries.
-

2. Learning Objectives

By completing this project, you will be able to:

- Use `CHECK` constraints and `ENUM`-like `VARCHAR` constraints for status columns.
 - Design an audit ledger table for tracking inventory movements.
 - Implement FIFO-style costing logic using window functions.
 - Use `MATERIALIZED VIEW` for expensive inventory valuation reports.
 - Write queries that aggregate across multiple warehouse locations.
 - Implement low-stock alerting with threshold-based queries.
 - Use `FOR UPDATE` locking in stock adjustment transactions.
 - Practice `UPSERT` (`INSERT ... ON CONFLICT DO UPDATE`) for stock reconciliation.
-

3. Functional Requirements

- **Products:** Catalog with SKU, name, category, unit of measure, cost price, and selling price.
 - **Suppliers:** Track suppliers with contact info and lead times.
 - **Warehouses:** Multiple storage locations, each tracking stock independently.
 - **Stock Levels:** Real-time available quantity per product per warehouse.
 - **Stock Movements:** Every stock change recorded as a movement event (received, sold, adjusted, transferred).
 - **Purchase Orders:** Create POs to suppliers, receive goods, auto-update stock.
 - **Low-Stock Alerts:** Identify products below reorder point.
 - **Inventory Valuation:** Calculate total inventory value using average cost method.
-

4. Non-Functional Requirements

- SKU must be unique and follow format `[A-Z]{2,4}-[0-9]{4,8}`.
- Quantity cannot go below zero (enforced at trigger level).
- All price/cost columns use `NUMERIC(12,2)`.

- Stock adjustments require a reason code.
- Purchase order status must follow: Draft -> Sent -> Partial -> Received -> Cancelled.

5. Complete Database Schema

```
-- =====
-- INVENTORY MANAGEMENT SYSTEM - Complete Schema
-- PostgreSQL 15+
-- =====

CREATE SCHEMA IF NOT EXISTS inv;
SET search_path = inv, public;

-- -----
-- CATEGORIES (hierarchical)
-- -----

CREATE TABLE categories (
    category_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE,
    parent_id INTEGER REFERENCES categories(category_id),
    description TEXT
);

INSERT INTO categories (name) VALUES
    ('Electronics'),('Clothing'),('Food & Beverage'),
    ('Office Supplies'),('Furniture'),('Tools & Hardware');

INSERT INTO categories (name, parent_id) VALUES
    ('Laptops', 1),('Smartphones', 1),('Headphones', 1),
    ('Men's Apparel', 2),('Women's Apparel', 2);

-- -----
-- UNITS OF MEASURE
-- -----

CREATE TABLE units (
    unit_id SERIAL PRIMARY KEY,
    code VARCHAR(10) NOT NULL UNIQUE,
    name VARCHAR(50) NOT NULL,
    description VARCHAR(200)
);

INSERT INTO units (code, name) VALUES
    ('EACH','Each'),('KG','Kilogram'),('LB','Pound'),
    ('PACK','Pack'),('BOX','Box'),('LITRE','Litre');

-- -----
-- SUPPLIERS
-- -----

CREATE TABLE suppliers (
    supplier_id SERIAL PRIMARY KEY,
```

```

code          VARCHAR(20) NOT NULL UNIQUE,
name          VARCHAR(200) NOT NULL,
contact_name  VARCHAR(200),
email         VARCHAR(300),
phone         VARCHAR(30),
address       TEXT,
country       VARCHAR(100) NOT NULL DEFAULT 'USA',
lead_time_days SMALLINT NOT NULL DEFAULT 7 CHECK (lead_time_days >= 0),
payment_terms VARCHAR(50) DEFAULT 'Net30',
is_active     BOOLEAN NOT NULL DEFAULT TRUE,
created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-----
-- PRODUCTS
-----

CREATE TABLE products (
    product_id SERIAL PRIMARY KEY,
    sku         VARCHAR(30) NOT NULL UNIQUE
               CHECK (sku ~ '^[A-Z]{2,4}-[0-9]{4,8}$'),
    name        VARCHAR(300) NOT NULL,
    category_id INTEGER REFERENCES categories(category_id),
    unit_id     INTEGER NOT NULL REFERENCES units(unit_id),
    supplier_id INTEGER REFERENCES suppliers(supplier_id),
    cost_price  NUMERIC(12,2) NOT NULL DEFAULT 0.00 CHECK (cost_price >= 0),
    sell_price  NUMERIC(12,2) NOT NULL DEFAULT 0.00 CHECK (sell_price >= 0),
    weight_kg   NUMERIC(8,3),
    barcode     VARCHAR(100),
    description TEXT,
    is_active   BOOLEAN NOT NULL DEFAULT TRUE,
    is_serialized BOOLEAN NOT NULL DEFAULT FALSE,
    reorder_point INTEGER NOT NULL DEFAULT 10 CHECK (reorder_point >= 0),
    reorder_quantity INTEGER NOT NULL DEFAULT 50 CHECK (reorder_quantity > 0),
    created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

COMMENT ON COLUMN products.reorder_point IS 'Trigger PO when stock falls to this level';
COMMENT ON COLUMN products.reorder_quantity IS 'Default qty to order when reorder triggered';

CREATE INDEX idx_products_sku ON products (sku);
CREATE INDEX idx_products_category ON products (category_id);
CREATE INDEX idx_products_supplier ON products (supplier_id);
CREATE INDEX idx_products_active ON products (is_active) WHERE is_active = TRUE;

-----
-- WAREHOUSES
-----

CREATE TABLE warehouses (
    warehouse_id SERIAL PRIMARY KEY,

```

```

code          VARCHAR(20) NOT NULL UNIQUE,
name          VARCHAR(200) NOT NULL,
address       TEXT,
city          VARCHAR(100),
country       VARCHAR(100) NOT NULL DEFAULT 'USA',
manager_name  VARCHAR(200),
is_active     BOOLEAN NOT NULL DEFAULT TRUE
);

INSERT INTO warehouses (code, name, city) VALUES
('WH-MAIN', 'Main Warehouse', 'Chicago'),
('WH-EAST', 'East Coast Depot', 'New York'),
('WH-WEST', 'West Coast Depot', 'Los Angeles');

-- -----
-- STOCK LEVELS (current state)
-- -----

CREATE TABLE stock_levels (
    stock_id      SERIAL PRIMARY KEY,
    product_id    INTEGER NOT NULL REFERENCES products(product_id),
    warehouse_id  INTEGER NOT NULL REFERENCES warehouses(warehouse_id),
    qty_on_hand   INTEGER NOT NULL DEFAULT 0 CHECK (qty_on_hand >= 0),
    qty_reserved  INTEGER NOT NULL DEFAULT 0 CHECK (qty_reserved >= 0),
    avg_cost      NUMERIC(12,4) NOT NULL DEFAULT 0.0000,
    last_updated  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE (product_id, warehouse_id)
);

CREATE INDEX idx_stock_product  ON stock_levels (product_id);
CREATE INDEX idx_stock_warehouse ON stock_levels (warehouse_id);
CREATE INDEX idx_stock_low ON stock_levels (product_id) WHERE qty_on_hand = 0;

-- -----
-- STOCK MOVEMENTS (audit ledger)
-- -----

CREATE TABLE movement_types (
    type_id      SERIAL PRIMARY KEY,
    code         VARCHAR(20) NOT NULL UNIQUE,
    name         VARCHAR(100) NOT NULL,
    direction    SMALLINT NOT NULL CHECK (direction IN (-1, 0, 1)),
    -- -1 = decrease, 1 = increase, 0 = neutral (e.g., adjustment)
    description  TEXT
);

INSERT INTO movement_types (code, name, direction) VALUES
('RECEIVE', 'Stock Receipt', 1),
('SELL', 'Sale', -1),
('ADJUST_POS', 'Positive Adjustment', 1),
('ADJUST_NEG', 'Negative Adjustment', -1),
('TRANSFER_IN', 'Transfer In', 1),
('TRANSFER_OUT', 'Transfer Out', -1),
('RETURN', 'Customer Return', 1),

```

```

('WRITE_OFF', 'Write Off', -1),
('OPENING', 'Opening Balance', 1);

CREATE TABLE stock_movements (
    movement_id BIGSERIAL PRIMARY KEY,
    product_id INTEGER NOT NULL REFERENCES products(product_id),
    warehouse_id INTEGER NOT NULL REFERENCES warehouses(warehouse_id),
    type_id INTEGER NOT NULL REFERENCES movement_types(type_id),
    qty INTEGER NOT NULL CHECK (qty > 0),
    unit_cost NUMERIC(12,4),
    reference_id INTEGER, -- PO, sale order, transfer ID
    reference_type VARCHAR(30), -- 'PO', 'SALE', 'TRANSFER'
    notes TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    created_by VARCHAR(100)
);

COMMENT ON TABLE stock_movements IS 'Immutable ledger of all inventory changes';
CREATE INDEX idx_movements_product ON stock_movements (product_id, created_at
DESC);
CREATE INDEX idx_movements_warehouse ON stock_movements (warehouse_id);
CREATE INDEX idx_movements_reference ON stock_movements (reference_type,
reference_id);
CREATE INDEX idx_movements_date ON stock_movements (created_at);

-----
-- PURCHASE ORDERS
-----

CREATE TABLE purchase_orders (
    po_id SERIAL PRIMARY KEY,
    po_number VARCHAR(30) NOT NULL UNIQUE,
    supplier_id INTEGER NOT NULL REFERENCES suppliers(supplier_id),
    warehouse_id INTEGER NOT NULL REFERENCES warehouses(warehouse_id),
    status VARCHAR(20) NOT NULL DEFAULT 'Draft'
    CHECK (status IN
('Draft', 'Sent', 'Partial', 'Received', 'Cancelled')),
    ordered_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    expected_by DATE,
    received_at TIMESTAMPTZ,
    total_amount NUMERIC(14,2),
    notes TEXT
);

CREATE INDEX idx_po_supplier ON purchase_orders (supplier_id);
CREATE INDEX idx_po_status ON purchase_orders (status) WHERE status NOT IN
('Received', 'Cancelled');

CREATE TABLE po_line_items (
    line_id SERIAL PRIMARY KEY,
    po_id INTEGER NOT NULL REFERENCES purchase_orders(po_id) ON DELETE
CASCADE,
    product_id INTEGER NOT NULL REFERENCES products(product_id),

```

```

    qty_ordered  INTEGER NOT NULL CHECK (qty_ordered > 0),
    qty_received INTEGER NOT NULL DEFAULT 0 CHECK (qty_received >= 0),
    unit_cost    NUMERIC(12,4) NOT NULL,
    UNIQUE (po_id, product_id)
);

-----
-- ADJUSTMENT REASONS
-----

CREATE TABLE adjustment_reasons (
    reason_id SERIAL PRIMARY KEY,
    code      VARCHAR(30) NOT NULL UNIQUE,
    name      VARCHAR(100) NOT NULL
);

INSERT INTO adjustment_reasons (code, name) VALUES
('DAMAGED',    'Damaged Goods'),
('COUNT_ADJ', 'Physical Count Adjustment'),
('EXPIRED',    'Expired Product'),
('LOST',       'Lost/Missing Stock'),
('SAMPLE',     'Sample Used'),
('OPENING_BAL', 'Opening Balance Setup');

```

6. ASCII ER Diagram

```

CATEGORIES (self-ref)
+-----+
| category_id(PK) |
| name (UNIQUE)   |<--+
| parent_id(FK)   |---+ (hierarchy)
+-----+-----+
      |
      v
    PRODUCTS
+-----+-----+
| product_id (PK) |
| sku (UNIQUE)    |
| name            |
| category_id (FK)|
| unit_id (FK)    |<---UNITS
| supplier_id (FK)|<---SUPPLIERS
| cost_price      | +-----+
| reorder_point   | | supplier_id (PK) |
+-----+-----+ | code (UNIQUE)   |
      |           | lead_time_days  |
      +-----+ +-----+
      |           |           |
      v           v           v
STOCK_LEVELS  STOCK_MOVEMENTS  PURCHASE_ORDERS
+-----+ +-----+ +-----+

```

stock_id	movement_id	po_id (PK)
product_id	product_id	po_number
warehouse_id	warehouse_id	supplier_id
qty_on_hand	type_id(FK)	warehouse_id
avg_cost	qty	status
+-----+	unit_cost	+-----+
	reference_id	
	+-----+	v
		PO_LINE_ITEMS
MOVEMENT_TYPES	WAREHOUSES	+-----+
+-----+	+-----+	line_id
type_id	warehouse_id	po_id (FK)
code	code	product_id
direction	name	qty_ordered
+-----+	+-----+	qty_received
		+-----+

7. Sample Data Scripts

```

SET search_path = inv, public;

-- Suppliers
INSERT INTO suppliers (code, name, contact_name, email, lead_time_days, country)
VALUES
  ('SUP-001', 'TechGadgets Co', 'John Lee', 'jlee@techgadgets.com', 10,
'USA'),
  ('SUP-002', 'Global Parts Ltd', 'Maria Chen', 'mchen@globalparts.com', 14,
'China'),
  ('SUP-003', 'Office World Inc', 'Tom Harris', 'tharris@officeworld.com', 5,
'USA'),
  ('SUP-004', 'Fashion Forward', 'Lisa Park', 'lpark@fashionforward.com', 21,
'Vietnam');

-- Products
INSERT INTO products (sku, name, category_id, unit_id, supplier_id, cost_price,
sell_price, reorder_point, reorder_quantity)
VALUES
  ('ELEC-00001', 'Wireless Headphones Pro', 3, 1, 1, 45.00, 129.99, 20, 50),
  ('ELEC-00002', 'USB-C Hub 7-Port', 1, 1, 2, 12.50, 34.99, 30, 100),
  ('ELEC-00003', 'Mechanical Keyboard RGB', 1, 1, 1, 35.00, 89.99, 15, 40),
  ('CLTH-00001', 'Classic Polo Shirt - Blue', 10, 1, 4, 8.00, 29.99, 50, 200),
  ('CLTH-00002', 'Running Shorts - Black', 10, 1, 4, 6.50, 24.99, 40, 150),
  ('OFFC-00001', 'Premium Ballpoint Pen Box', 4, 4, 3, 4.00, 11.99, 100, 500);

-- Stock levels (opening balances)
INSERT INTO stock_levels (product_id, warehouse_id, qty_on_hand, avg_cost) VALUES
  (1, 1, 150, 45.00), (1, 2, 50, 45.00), (1, 3, 30, 45.00),
  (2, 1, 300, 12.50), (2, 2, 100, 12.50),
  (3, 1, 120, 35.00), (3, 3, 45, 35.00),
  (4, 1, 500, 8.00), (4, 2, 200, 8.00),

```

```

(5, 1, 400, 6.50),
(6, 1, 1000, 4.00), (6, 2, 200, 4.00);

-- Stock movements (opening balances)
INSERT INTO stock_movements (product_id, warehouse_id, type_id, qty, unit_cost,
reference_type, notes)
SELECT sl.product_id, sl.warehouse_id, 9, sl.qty_on_hand, sl.avg_cost, 'OPENING',
'Initial stock setup'
FROM stock_levels sl;

-- Purchase Orders
INSERT INTO purchase_orders (po_number, supplier_id, warehouse_id, status,
expected_by)
VALUES
('PO-2024-001', 1, 1, 'Sent', CURRENT_DATE + 10),
('PO-2024-002', 2, 1, 'Draft', CURRENT_DATE + 14),
('PO-2024-003', 3, 2, 'Received', CURRENT_DATE - 5);

INSERT INTO po_line_items (po_id, product_id, qty_ordered, qty_received, unit_cost)
VALUES
(1, 1, 100, 0, 45.00),
(1, 2, 200, 0, 12.50),
(3, 6, 500, 500, 4.00);

```

8. Core SQL Queries

```

SET search_path = inv, public;

-- -----
-- Q1: Current stock levels across all warehouses
-- -----

SELECT
    p.sku,
    p.name,
    w.code AS warehouse,
    sl.qty_on_hand,
    sl.qty_reserved,
    sl.qty_on_hand - sl.qty_reserved AS qty_available,
    ROUND(sl.qty_on_hand * sl.avg_cost, 2) AS stock_value
FROM stock_levels sl
JOIN products p ON sl.product_id = p.product_id
JOIN warehouses w ON sl.warehouse_id = w.warehouse_id
WHERE p.is_active = TRUE
ORDER BY p.sku, w.code;

-- -----
-- Q2: Total inventory value by category
-- -----

SELECT
    c.name AS category,

```



```

COUNT(DISTINCT p.product_id)      AS product_count,
SUM(sl.qty_on_hand)                  AS total_units,
ROUND(SUM(sl.qty_on_hand * sl.avg_cost), 2) AS total_value
FROM stock_levels sl
JOIN products p    ON sl.product_id = p.product_id
JOIN categories c  ON p.category_id = c.category_id
GROUP BY c.category_id, c.name
ORDER BY total_value DESC;

```

```

-- -----
-- Q3: Low-stock alert - products below reorder point
-- -----

```

```

SELECT
    p.sku,
    p.name,
    w.name                        AS warehouse,
    sl.qty_on_hand                AS current_stock,
    p.reorder_point,
    p.reorder_quantity            AS suggested_order_qty,
    sup.name                      AS default_supplier,
    sup.lead_time_days
FROM stock_levels sl
JOIN products p    ON sl.product_id = p.product_id
JOIN warehouses w  ON sl.warehouse_id = w.warehouse_id
LEFT JOIN suppliers sup ON p.supplier_id = sup.supplier_id
WHERE sl.qty_on_hand <= p.reorder_point
      AND p.is_active = TRUE
ORDER BY sl.qty_on_hand - p.reorder_point;

```

```

-- -----
-- Q4: Stock movement history for a product
-- -----

```

```

SELECT
    sm.created_at::DATE      AS date,
    mt.name                  AS movement_type,
    w.code                   AS warehouse,
    CASE WHEN mt.direction = 1 THEN sm.qty
          WHEN mt.direction = -1 THEN -sm.qty
          ELSE 0
    END                      AS qty_change,
    sm.unit_cost,
    sm.reference_type,
    sm.notes
FROM stock_movements sm
JOIN movement_types mt ON sm.type_id = mt.type_id
JOIN warehouses w      ON sm.warehouse_id = w.warehouse_id
WHERE sm.product_id = 1
ORDER BY sm.created_at DESC;

```

```

-- -----
-- Q5: Running inventory balance per product (window function)
-- -----

```

```

SELECT
    sm.created_at::DATE AS date,
    mt.name                AS movement_type,
    CASE WHEN mt.direction = 1 THEN  sm.qty
         WHEN mt.direction = -1 THEN -sm.qty ELSE 0 END AS change,
    SUM(CASE WHEN mt.direction = 1 THEN  sm.qty
         WHEN mt.direction = -1 THEN -sm.qty
         ELSE 0 END)
        OVER (PARTITION BY sm.product_id, sm.warehouse_id
              ORDER BY sm.created_at
              ROWS UNBOUNDED PRECEDING) AS running_balance
FROM stock_movements sm
JOIN movement_types mt ON sm.type_id = mt.type_id
WHERE sm.product_id = 1 AND sm.warehouse_id = 1
ORDER BY sm.created_at;

```

```

-- -----
-- Q6: Open purchase orders with line items
-- -----

```

```

SELECT
    po.po_number,
    s.name                AS supplier,
    po.status,
    po.ordered_at::DATE,
    po.expected_by,
    p.sku,
    p.name                AS product,
    li.qty_ordered,
    li.qty_received,
    li.qty_ordered - li.qty_received AS qty_outstanding,
    ROUND(li.qty_ordered * li.unit_cost, 2) AS line_total
FROM purchase_orders po
JOIN suppliers s      ON po.supplier_id = s.supplier_id
JOIN po_line_items li ON po.po_id       = li.po_id
JOIN products p       ON li.product_id  = p.product_id
WHERE po.status NOT IN ('Received', 'Cancelled')
ORDER BY po.expected_by, po.po_number;

```

```

-- -----
-- Q7: Inventory valuation report using average cost
-- -----

```

```

WITH product_totals AS (
    SELECT
        p.product_id,
        p.sku,
        p.name,
        SUM(sl.qty_on_hand)                AS total_qty,
        ROUND(SUM(sl.qty_on_hand * sl.avg_cost), 2) AS total_cost_value,
        ROUND(SUM(sl.qty_on_hand * p.sell_price), 2) AS total_sell_value
    FROM stock_levels sl
    JOIN products p ON sl.product_id = p.product_id
    GROUP BY p.product_id, p.sku, p.name, p.sell_price
)

```

```

)
SELECT
    *,
    ROUND(total_sell_value - total_cost_value, 2) AS gross_margin,
    ROUND(100.0 * (total_sell_value - total_cost_value) / NULLIF(total_sell_value,
0), 1) AS margin_pct
FROM product_totals
ORDER BY total_cost_value DESC;

```

```

-- -----
-- Q8: Supplier performance summary
-- -----

```

```

SELECT
    s.name AS supplier,
    COUNT(DISTINCT po.po_id) AS total_orders,
    COUNT(DISTINCT po.po_id) FILTER (WHERE po.status = 'Received') AS
completed_orders,
    ROUND(AVG(po.received_at::DATE - po.ordered_at::DATE)
        FILTER (WHERE po.status = 'Received'), 1) AS avg_fulfillment_days,
    s.lead_time_days AS stated_lead_time
FROM suppliers s
LEFT JOIN purchase_orders po ON s.supplier_id = po.supplier_id
GROUP BY s.supplier_id, s.name, s.lead_time_days
ORDER BY completed_orders DESC;

```

```

-- -----
-- Q9: Products with zero stock in any warehouse
-- -----

```

```

SELECT
    p.sku,
    p.name,
    w.code AS warehouse,
    'OUT OF STOCK' AS status
FROM stock_levels sl
JOIN products p ON sl.product_id = p.product_id
JOIN warehouses w ON sl.warehouse_id = w.warehouse_id
WHERE sl.qty_on_hand = 0 AND p.is_active = TRUE;

```

```

-- -----
-- Q10: Stock transfer between warehouses
-- (analytical - what would need to move to balance stock)
-- -----

```

```

WITH warehouse_totals AS (
    SELECT
        p.sku,
        p.name,
        w.code AS warehouse,
        sl.qty_on_hand,
        AVG(sl.qty_on_hand) OVER (PARTITION BY p.product_id) AS avg_across_wh,
        sl.qty_on_hand - AVG(sl.qty_on_hand) OVER (PARTITION BY p.product_id)
            AS deviation_from_avg
    FROM stock_levels sl

```

```

        JOIN products p    ON sl.product_id = p.product_id
        JOIN warehouses w  ON sl.warehouse_id = w.warehouse_id
    )
SELECT *
FROM warehouse_totals
WHERE ABS(deviation_from_avg) > 20
ORDER BY ABS(deviation_from_avg) DESC;

-- -----
-- Q11: Monthly stock receipts summary
-- -----

SELECT
    DATE_TRUNC('month', sm.created_at)::DATE AS month,
    p.sku,
    p.name,
    SUM(sm.qty)                               AS qty_received,
    ROUND(SUM(sm.qty * sm.unit_cost), 2)      AS total_cost
FROM stock_movements sm
JOIN products p      ON sm.product_id = p.product_id
JOIN movement_types mt ON sm.type_id = mt.type_id
WHERE mt.code = 'RECEIVE'
GROUP BY 1, p.product_id, p.sku, p.name
ORDER BY 1 DESC, total_cost DESC;

-- -----
-- Q12: Dead stock - products not moved in 90+ days
-- -----

SELECT
    p.sku,
    p.name,
    MAX(sm.created_at)::DATE      AS last_movement,
    CURRENT_DATE - MAX(sm.created_at)::DATE AS days_since_movement,
    SUM(sl.qty_on_hand)           AS current_stock,
    ROUND(SUM(sl.qty_on_hand * sl.avg_cost), 2) AS stock_value
FROM products p
LEFT JOIN stock_movements sm ON p.product_id = sm.product_id
JOIN stock_levels sl        ON p.product_id = sl.product_id
WHERE p.is_active = TRUE
GROUP BY p.product_id, p.sku, p.name
HAVING MAX(sm.created_at) < NOW() - INTERVAL '90 days'
      OR MAX(sm.created_at) IS NULL
ORDER BY days_since_movement DESC NULLS FIRST;

-- -----
-- Q13: Reorder suggestions with estimated PO value
-- -----

WITH stock_totals AS (
    SELECT product_id, SUM(qty_on_hand) AS total_on_hand
    FROM stock_levels
    GROUP BY product_id
)
SELECT

```

```

    p.sku,
    p.name,
    st.total_on_hand                                AS current_stock,
    p.reorder_point,
    p.reorder_quantity,
    ROUND(p.reorder_quantity * p.cost_price, 2)      AS estimated_po_value,
    sup.name                                         AS supplier,
    sup.lead_time_days,
    (CURRENT_DATE + sup.lead_time_days)::DATE        AS expected_arrival
FROM products p
JOIN stock_totals st ON p.product_id = st.product_id
LEFT JOIN suppliers sup ON p.supplier_id = sup.supplier_id
WHERE st.total_on_hand <= p.reorder_point
      AND p.is_active = TRUE
ORDER BY estimated_po_value DESC;

-- -----
-- Q14: ABC Analysis (A=top 80%, B=next 15%, C=bottom 5%)
-- -----

WITH values AS (
    SELECT
        p.product_id,
        p.sku,
        p.name,
        ROUND(SUM(sl.qty_on_hand * sl.avg_cost), 2) AS stock_value
    FROM stock_levels sl
    JOIN products p ON sl.product_id = p.product_id
    GROUP BY p.product_id, p.sku, p.name
),
totals AS (
    SELECT SUM(stock_value) AS grand_total FROM values
),
ranked AS (
    SELECT v.*,
        ROUND(100.0 * v.stock_value / t.grand_total, 2) AS value_pct,
        ROUND(100.0 * SUM(v.stock_value) OVER (ORDER BY v.stock_value DESC)
              / t.grand_total, 2) AS
cumulative_pct
    FROM values v CROSS JOIN totals t
)
SELECT
    sku, name, stock_value, value_pct, cumulative_pct,
    CASE WHEN cumulative_pct <= 80 THEN 'A'
         WHEN cumulative_pct <= 95 THEN 'B'
         ELSE 'C'
    END AS abc_class
FROM ranked
ORDER BY stock_value DESC;

-- -----
-- Q15: Warehouse utilization comparison
-- -----

```

```

SELECT
    w.name                                AS warehouse,
    COUNT(DISTINCT sl.product_id)         AS product_count,
    SUM(sl.qty_on_hand)                   AS total_units,
    ROUND(SUM(sl.qty_on_hand * sl.avg_cost), 2) AS total_value,
    ROUND(100.0 * SUM(sl.qty_on_hand * sl.avg_cost)
          / SUM(SUM(sl.qty_on_hand * sl.avg_cost)) OVER (), 1) AS value_pct
FROM stock_levels sl
JOIN warehouses w ON sl.warehouse_id = w.warehouse_id
GROUP BY w.warehouse_id, w.name
ORDER BY total_value DESC;

```

9. Stored Procedures / Functions

```

-- -----
-- FUNCTION 1: Record a stock movement and update stock level
-- -----

CREATE OR REPLACE FUNCTION inv.record_movement(
    p_product_id    INTEGER,
    p_warehouse_id  INTEGER,
    p_type_code      VARCHAR,
    p_qty            INTEGER,
    p_unit_cost      NUMERIC(12,4) DEFAULT NULL,
    p_reference_id   INTEGER DEFAULT NULL,
    p_reference_type VARCHAR DEFAULT NULL,
    p_notes          TEXT DEFAULT NULL,
    p_user           VARCHAR DEFAULT 'system'
)
RETURNS BIGINT
LANGUAGE plpgsql AS $$
DECLARE
    v_type      inv.movement_types%ROWTYPE;
    v_move_id   BIGINT;
    v_new_qty    INTEGER;
    v_new_cost   NUMERIC(12,4);
    v_curr      inv.stock_levels%ROWTYPE;
BEGIN
    SELECT * INTO v_type FROM inv.movement_types WHERE code = p_type_code;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'Unknown movement type: %', p_type_code;
    END IF;

    -- Lock stock level row
    SELECT * INTO v_curr FROM inv.stock_levels
    WHERE product_id = p_product_id AND warehouse_id = p_warehouse_id
    FOR UPDATE;

    IF NOT FOUND THEN
        -- Auto-create stock level entry if missing
        INSERT INTO inv.stock_levels (product_id, warehouse_id, qty_on_hand,

```

```

avg_cost)
    VALUES (p_product_id, p_warehouse_id, 0, 0)
    RETURNING * INTO v_curr;
END IF;

-- Calculate new quantity
v_new_qty := v_curr.qty_on_hand + (v_type.direction * p_qty);
IF v_new_qty < 0 THEN
    RAISE EXCEPTION 'Insufficient stock. Current: %, Requested decrease: %',
        v_curr.qty_on_hand, p_qty;
END IF;

-- Update moving average cost on receipts
IF v_type.direction = 1 AND p_unit_cost IS NOT NULL THEN
    v_new_cost := (
        (v_curr.qty_on_hand * v_curr.avg_cost + p_qty * p_unit_cost)
        / NULLIF(v_curr.qty_on_hand + p_qty, 0)
    );
ELSE
    v_new_cost := v_curr.avg_cost;
END IF;

-- Insert movement record
INSERT INTO inv.stock_movements (
    product_id, warehouse_id, type_id, qty, unit_cost,
    reference_id, reference_type, notes, created_by
)
VALUES (
    p_product_id, p_warehouse_id, v_type.type_id, p_qty,
    COALESCE(p_unit_cost, v_curr.avg_cost),
    p_reference_id, p_reference_type, p_notes, p_user
)
RETURNING movement_id INTO v_move_id;

-- Update stock level
UPDATE inv.stock_levels
SET qty_on_hand = v_new_qty,
    avg_cost = COALESCE(v_new_cost, avg_cost),
    last_updated = NOW()
WHERE product_id = p_product_id AND warehouse_id = p_warehouse_id;

RETURN v_move_id;
END;
$$;

-- -----
-- FUNCTION 2: Receive a purchase order
-- -----
CREATE OR REPLACE FUNCTION inv.receive_purchase_order(
    p_po_id    INTEGER,
    p_user     VARCHAR DEFAULT 'warehouse_staff'
)

```

```

RETURNS TEXT
LANGUAGE plpgsql AS $$
DECLARE
    v_po          inv.purchase_orders%ROWTYPE;
    v_line        inv.po_line_items%ROWTYPE;
    v_received    INTEGER := 0;
BEGIN
    SELECT * INTO v_po FROM inv.purchase_orders WHERE po_id = p_po_id FOR UPDATE;
    IF NOT FOUND THEN RAISE EXCEPTION 'PO % not found', p_po_id; END IF;
    IF v_po.status IN ('Received', 'Cancelled') THEN
        RAISE EXCEPTION 'PO % is already % and cannot be received again', p_po_id,
v_po.status;
    END IF;

    FOR v_line IN
        SELECT * FROM inv.po_line_items
        WHERE po_id = p_po_id AND qty_ordered > qty_received
    LOOP
        PERFORM inv.record_movement(
            v_line.product_id,
            v_po.warehouse_id,
            'RECEIVE',
            v_line.qty_ordered - v_line.qty_received,
            v_line.unit_cost,
            p_po_id,
            'PO',
            'Received from PO ' || p_po_id,
            p_user
        );

        UPDATE inv.po_line_items
        SET qty_received = qty_ordered
        WHERE line_id = v_line.line_id;

        v_received := v_received + 1;
    END LOOP;

    UPDATE inv.purchase_orders
    SET status = 'Received', received_at = NOW()
    WHERE po_id = p_po_id;

    RETURN format('PO %s received. %s line items processed.', p_po_id, v_received);
END;
$$;

-- -----
-- FUNCTION 3: Transfer stock between warehouses
-- -----

CREATE OR REPLACE FUNCTION inv.transfer_stock(
    p_product_id    INTEGER,
    p_from_warehouse INTEGER,
    p_to_warehouse  INTEGER,

```



```

        p_qty          INTEGER,
        p_user         VARCHAR DEFAULT 'system'
    )
    RETURNS TEXT
    LANGUAGE plpgsql AS $$
    BEGIN
        IF p_from_warehouse = p_to_warehouse THEN
            RAISE EXCEPTION 'Source and destination warehouses must be different';
        END IF;
        IF p_qty <= 0 THEN RAISE EXCEPTION 'Transfer quantity must be positive'; END IF;

        PERFORM inv.record_movement(p_product_id, p_from_warehouse, 'TRANSFER_OUT',
        p_qty,
                                NULL, NULL, 'TRANSFER', 'Transfer to WH ' ||
        p_to_warehouse, p_user);
        PERFORM inv.record_movement(p_product_id, p_to_warehouse, 'TRANSFER_IN',
        p_qty,
                                (SELECT avg_cost FROM inv.stock_levels
                                WHERE product_id = p_product_id AND warehouse_id =
        p_from_warehouse),
                                NULL, 'TRANSFER', 'Transfer from WH ' ||
        p_from_warehouse, p_user);

        RETURN format('Transferred %s units of product %s from WH %s to WH %s',
            p_qty, p_product_id, p_from_warehouse, p_to_warehouse);
    END;
    $$;

```

10. Triggers

```

-- -----
-- TRIGGER 1: Prevent direct updates to stock_levels qty
-- (force all changes through record_movement function)
-- -----
CREATE OR REPLACE FUNCTION inv.trg_protect_stock_levels()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    IF TG_OP = 'UPDATE'
        AND (NEW.qty_on_hand <> OLD.qty_on_hand OR NEW.avg_cost <> OLD.avg_cost)
        AND current_setting('inv.allow_direct_update', TRUE) <> 'true' THEN
        RAISE EXCEPTION 'Direct stock_levels updates are not allowed. Use
inv.record_movement().';
    END IF;
    NEW.last_updated := NOW();
    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_stock_levels_protect
BEFORE UPDATE ON inv.stock_levels

```

```

    FOR EACH ROW
    EXECUTE FUNCTION inv.trg_protect_stock_levels();

-----
-- TRIGGER 2: Update product updated_at timestamp
-----

CREATE OR REPLACE FUNCTION inv.trg_products_timestamp()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    NEW.updated_at := NOW();
    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_products_updated
BEFORE UPDATE ON inv.products
FOR EACH ROW
EXECUTE FUNCTION inv.trg_products_timestamp();

```

11. Performance Optimization

```

-- Materialized view for fast inventory dashboard
CREATE MATERIALIZED VIEW inv.mv_inventory_summary AS
SELECT
    p.product_id,
    p.sku,
    p.name,
    c.name          AS category,
    SUM(sl.qty_on_hand)          AS total_qty,
    ROUND(SUM(sl.qty_on_hand * sl.avg_cost), 2)  AS total_cost_value,
    ROUND(SUM(sl.qty_on_hand * p.sell_price), 2)  AS total_sell_value,
    p.reorder_point,
    CASE WHEN SUM(sl.qty_on_hand) <= p.reorder_point THEN TRUE ELSE FALSE END AS
needs_reorder
FROM stock_levels sl
JOIN products p    ON sl.product_id = p.product_id
LEFT JOIN categories c ON p.category_id = c.category_id
GROUP BY p.product_id, p.sku, p.name, c.name, p.sell_price, p.reorder_point
WITH DATA;

CREATE UNIQUE INDEX ON inv.mv_inventory_summary (product_id);
CREATE INDEX ON inv.mv_inventory_summary (needs_reorder) WHERE needs_reorder = TRUE;

-- Refresh the materialized view (can be scheduled with pg_cron)
-- REFRESH MATERIALIZED VIEW CONCURRENTLY inv.mv_inventory_summary;

```

Index	Reason
idx_movements_product (composite)	Fast history per product ordered by date

idx_movements_date	Time-range queries on movements ledger
idx_stock_low (partial)	Instant low-stock detection
Materialized view	Expensive inventory valuation cached

12. Extensions Used

```
CREATE EXTENSION IF NOT EXISTS pg_cron;      -- Schedule materialized view refreshes
CREATE EXTENSION IF NOT EXISTS pg_trgm;      -- Fuzzy product search by name

-- Schedule nightly refresh of inventory summary
SELECT cron.schedule('refresh-inv-summary', '0 2 * * *',
    'REFRESH MATERIALIZED VIEW CONCURRENTLY inv.mv_inventory_summary');
```

13. Testing Guide

```
-- TEST 1: Record a sale movement
SELECT inv.record_movement(1, 1, 'SELL', 10, NULL, 101, 'SALE', 'Sale order 101');
SELECT qty_on_hand FROM inv.stock_levels WHERE product_id = 1 AND warehouse_id = 1;
-- Should be 140

-- TEST 2: Try to sell more than available
SELECT inv.record_movement(1, 1, 'SELL', 9999, NULL);
-- Should raise "Insufficient stock" error

-- TEST 3: Receive a purchase order
SELECT inv.receive_purchase_order(1);
SELECT qty_on_hand FROM inv.stock_levels WHERE product_id = 1 AND warehouse_id = 1;
-- Should increase by PO quantity

-- TEST 4: Transfer stock
SELECT inv.transfer_stock(2, 1, 2, 50);
SELECT product_id, warehouse_id, qty_on_hand FROM inv.stock_levels WHERE product_id
= 2;
-- WH1 decreases, WH2 increases

-- TEST 5: Verify running balance (Q5)
-- Should see correct running totals for product 1, warehouse 1

-- TEST 6: Low-stock alert
UPDATE inv.stock_levels SET qty_on_hand = 5 WHERE product_id = 3 AND warehouse_id =
1;
-- Run Q3 and confirm product 3 appears
```

14. Extension Challenges

- 1. **Batch/Lot Tracking:** Add a `lots` table to track stock by batch/lot number and expiry date. FIFO picking logic selects oldest lot first. Write a function that picks quantities using FIFO.
- 2. **Sales Order Integration:** Build a `sales_orders` and `sales_order_lines` table. Reserving stock decrements `qty_reserved` , not `qty_on_hand` . Fulfilling the order triggers the SELL movement.
- 3. **Barcode Scanning Workflow:** Simulate a cycle count where barcodes are scanned. Write a function that compares scanned counts to expected and generates an adjustment movement for any discrepancy.
- 4. **Landed Cost Calculation:** Add freight, duty, and handling costs to purchase orders. Write a function that apportions landed costs across line items by weight and updates average cost accordingly.
- 5. **Demand Forecasting:** Build a `sales_history` table with daily/weekly sales. Write a query using moving averages to forecast demand for the next 30 days and auto-generate reorder suggestions.

15. What You Learned

Skill	Demonstrated By
Ledger/audit table design	Immutable <code>stock_movements</code> table
Moving average cost tracking	<code>avg_cost</code> recalculation on receipt
Materialized views	Inventory summary dashboard
UPSERT pattern	Stock level auto-creation in <code>record_movement</code>
Partial indexes	Low-stock and active product indexes
ABC analysis	Cumulative percentage window function
Hierarchical categories	Self-referential <code>categories</code> table
Transactional stock control	<code>FOR UPDATE</code> locking in movement function
Custom constraints	SKU format CHECK with regex